

Effects of Underflow on Solving Linear Systems (Excerpt)

James Demmel
Computer Science Division
Electronic Research Laboratory
U.C. Berkeley

May 1981

Abstract

<https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/CSD-83-128.pdf>

This is an excerpt, prepared by Andrew W. Appel in 2025, of some parts of the tech. report relevant to accuracy bounds for forward substitution and back substitution.

In this paper we examine the effects of underflow on solving systems of linear equations using Gaussian Elimination. Our goal is to decide if reliable software for solving linear systems in the presence of underflow can be written at reasonable cost. We contrast the utilities of gradual underflow and “store zero”, and show that only by using gradual underflow can we achieve reliability easily.

1. Introduction

...

2. Examples

...

Let ϵ be the rounding error of the arithmetic, and λ be the underflow threshold. Suppose, for example, a binary floating point number is represented as $f \cdot 2^e$ where f lies between 1 and 2 with an n bit fraction, and e is an integer exponent in the range $e_{\min} \leq e \leq e_{\max}$. Then $\epsilon = 2^{-n}$ (the difference between 1 and the next largest number). The underflow threshold is $\lambda = 2^{e_{\min}}$; this is the smallest ... normalized number for G.U. ... Denormalized numbers lie between 0 and $2^{e_{\min}}$ in G.U. arithmetic for which the smallest nonzero number is thus $\mu = \epsilon\lambda$. We will give a more precise model of rounding and underflow errors in the next section.

3. Error Analysis

...

Let $\blacksquare$ be one of the operations $(+, -, *, /)$ and let $fl(a \blacksquare b)$ denote the floating point result of the indicated computation. ... To take underflow into account, we write ([Kahan and Palmer, 1979])

$$fl(a \blacksquare b) = (a \blacksquare b)(1 + e) + \eta \quad \text{unless } a \blacksquare b \text{ overflows.} \quad (11)$$

In the case of G.U. we have the following constraints on e and η :

- (1) $|e| \leq \epsilon$ and $|\eta| \leq \lambda\epsilon$,
- (2) $\eta \times e = 0$
- (3) $\eta = 0$ if $\blacksquare$ is either addition or subtraction.

...

We define the function $UN(x)$ to be

$$UN(x) = \begin{cases} 1 & \text{if } x \text{ underflows } (x < \lambda) \\ 0 & \text{if it does not} \end{cases}$$

We assume no overflow occurs.

We ignore terms which are $O(\epsilon^2)$ or $O(\epsilon\mu)$. This allows us to make the following substitutions when convenient:

Replace $1/(1 + e_1)$ by $1 + e_2$,

Replace $(1 + e_1)^j$ by $1 + je_2$,

Replace $\prod_{j=1}^m (1 + e_j) / \prod_{i=1}^n (1 + e'_i)$ by $1 + (n + m)e$

Replace $\eta_1 * (1 + e_1)^j$ by η_2 .

By replacing every appearance of an original datum a by $1 \cdot a$, and using the above formula for error in multiplication, we can take into account effects of rounding and underflow errors in the original data.¹

$\|A\|$ and $\|b\|$ denote the norms of matrix A and vector b . $\|A\|_\infty$ denotes the infinity norm of A , and similarly for $\|b\|_\infty$. $k(A) = \|A\|_\infty \|A^{-1}\|_\infty$ denotes the condition number of the matrix A .

$|A|$ ($|b|$) denotes the matrix (vector) whose entries are the absolute values of the corresponding entries in A (b). Inequalities like $|A| > |B|$ ($|a| > |b|$) are meant componentwise.

3.2 Approach

As stated in the introduction, we use backward error analysis. Thus, when trying to solve

$$Ax = b \tag{12}$$

for x we obtain an approximation $\hat{x} = x + \delta x$ which satisfies the perturbed problem

$$(A + \delta A)\hat{x} = b + \delta b . \tag{13}$$

The task of backwards error analysis is to obtain bounds on δA and δb . These bounds can be used in turn to bound the residual r :

$$r = A\hat{x} - b = -\delta A\hat{x} + \delta b \tag{14}$$

...

3.3 Error Analysis Details

...

Proposition 2.

To analyze solving a triangular system, we need another expression for the error in computing an inner product:

$$\text{float}\left(\sum_{i=1}^n a_i b_i\right) = \sum_{i=1}^n a_i b_i (1 + E_i) + \eta . \tag{31}$$

In the absence of underflow we have

$$\begin{aligned} |E_1| &\leq n\epsilon \\ |E_j| &\leq (n + 2 - j)\epsilon \quad \text{if } j > 1 . \end{aligned} \tag{32}$$

In the case of G.U.² we have the same bounds on the $|E_i|$, and

$$|\eta| \leq n\lambda\epsilon \tag{34}$$

¹Editors note: Ugh. It's provable that $1 \cdot a = a$ exactly, so this notational trick does not appeal to me.

²Editor's note: G.U. = Gradual Underflow, that is, a floating-point format that supports subnormal numbers. The technical report also covers the case of "S.Z." or "store zero", which we omit here in all lemmas and theorems.

Proof. Letting

$$\begin{aligned}
S_m &= fl\left(\sum_{i=1}^m a_i b_i\right) \\
&= fl(S_{m-1} + a_m b_m) \\
&= (S_{m-1} + a_m b_m(1 + e_m) + \eta_m)(1 + w_{m-1}) + z_{m-1}
\end{aligned}$$

It is easy to prove by induction on m that

$$\begin{aligned}
S_m &= a_1 b_1(1 + e_1) \prod_{i=1}^{m-1} (1 + w_i) + \eta_1 \prod_{i=1}^{m-1} (1 + w_i) + \\
&\quad \sum_{k=2}^m a_k b_k(1 + e_k) \prod_{i=k-1}^{m-1} (1 + w_i) + \sum_{k=2}^m \eta_k \prod_{i=k-1}^{m-1} (1 + w_i) + \\
&\quad \sum_{k=1}^{m-1} z_k \prod_{i=k+1}^{m-1} (1 + w_i).
\end{aligned}$$

$z_{m-1} = 0$ always for G.U. ... Thus ... $z_{m-1} \eta_m = 0$, so the contribution from underflow is at most n times the bound for η_m and z_{m-1} . Approximating $(1 + e_k) \prod_{i=r}^{n-1} (1 + w_i)$ by $1 + (n - r + 1)\epsilon$ we get the result. Note that the $\sum_{i=1}^n a_i b_i(1 + E_i)$ term is the same term as would have been given by round-off alone. Q.E.D.

...

Lemma 4: Error Analysis of Solving a Triangular System

Consider the lower triangular system $Ly = b$ where L does not necessarily have ones on the diagonal. (The analyses when only ones are on the diagonal or L is upper triangular are similar.) Then in solving $Ly = b$ using the program in Appendix 2 (where inner products are accumulated with roundoff error ϵ_s and smallest number μ_s we obtain $\hat{y} = y + \delta y$ which actually satisfies $(L + \delta L)\hat{y} = b + \delta b$, where

$$|\delta L_{ij}| \leq |L_{ij}| \cdot \begin{cases} \epsilon & \text{if } i = j = 1 \\ (i-1)\epsilon_s & \text{if } i > j = 1 \\ (i+1-j)\epsilon_s & \text{if } i > j > 1 \\ \epsilon + \epsilon_s & \text{if } i = j > 1 \end{cases} \quad (80)$$

independent of underflow.

In the absence of underflow

$$\delta b = 0. \quad (81)$$

Using G.U.

$$\begin{aligned}
&\text{if } i = 1 \quad |\delta b_i| \leq \lambda \epsilon |L_{11}| UN(y_1) \quad \text{if } y_1 \text{ underflows} \\
&\text{if } i > 1 \quad |\delta b_i| \leq i \cdot \lambda_s \epsilon_s + \lambda \epsilon |L_{ii}| UN(y_i) \quad \text{if } y_i \text{ underflows} \\
&\quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \quad \text{intermediate underflows} \quad . \quad (82)
\end{aligned}$$

If L_{ii} is known to be 1, the only changes in the above bounds (besides substituting 1 for L_{ii}) are that $E_{11} = \delta b_1 = 0$ independent of underflow.

Proof: We apply Proposition 2 to the program in Appendix 2. We have two cases.

Case $i = 1$: In this case,

$$y_1 = float(b_1/L_{11}) = \frac{b_1}{L_{11}(1 + E_{11})} + \nu_1 \quad (84)$$

so

$$L_{11}(1 + E_{11})y_1 = b_1 + \nu_1 L_{11}(1 + E_{11}) UN(y_1) \quad (85)$$

where

$$|E_{11}| \leq \epsilon \quad \text{and} \quad |\nu_1| \leq \lambda \text{ or } \lambda/\epsilon \quad (86)$$

Case $i > 1$: Now

$$\begin{aligned} y_i &= \text{float}((b_i - \sum_{j=1}^{i-1} L_{ij}y_j)/L_{ii}) \\ &= [(1 + e_i)(b_i - \sum_{j=1}^{i-1} L_{ij}(1 + E_{ij})y_j - \rho_i) + \sigma_i]/(L_{ii}(1 + \zeta_i)) + \nu_i \end{aligned} \quad (87)$$

where

$$|E_{ij}| \leq \begin{cases} (i-1)\epsilon_s & \text{if } j = 1 \\ (i+1-j)\epsilon_s & \text{if } j > 1 \end{cases} \quad (88a)$$

$$|e_i| \leq \epsilon_s \quad (88b)$$

$$|\zeta_i| \leq \epsilon \quad (88c)$$

and

$$|\rho_i| \leq (i-1)\lambda_s\epsilon_s \quad \text{using G.U.} \quad (89a)$$

$$|\sigma_i| \leq \lambda_s\epsilon_s \quad \text{using G.U.} \quad (89b)$$

$$|\nu_i| \leq \lambda\epsilon \quad \text{using G.U.} \quad (89c)$$

In the absence of underflow, $\rho_i = \sigma_i = \nu_i = 0$. Rearranging equation (87), we obtain

$$L_{ii}(1 + \zeta_i)y_i = (1 + e_i)(b_i - \sum_{j=1}^{i-1} L_{ij}(1 + E_{ij})y_j - \rho_i) + \sigma_i + \nu_i L_{ii}(1 + \zeta_i) \quad (90)$$

or

$$b_i = \sum_{j=1}^{i-1} L_{ij}(1 + E_{ij})y_j + L_{ii} \left(\frac{1 + \zeta_i}{1 + e_i} \right) y_i + \rho_i - \frac{\sigma_i}{1 + e_i} - \nu_i L_{ii} \left(\frac{1 + \zeta_i}{1 + e_i} \right) . \quad (91)$$

Using the bounds in (88) and (89) in (91), we obtain the bounds in the statement of the lemma. Q.E.D.

...

Appendix 2: Program Listing of Solution of Both Triangular Systems Following Triangular Decomposition

```

array B[1..N] of real;
array X[1..N] of real;
array Y[1..N] of real;
/* Again, B,X, and Y could occupy the same locations in memory. Arrays
A,L and U are defined above. As in Appendix 1, precisions of all computations
except entries of B, X, and Y are shown. */

/* Forward Substitution: LY = B */

for i=1 to N do
  begin
    SUM=0;
    for k=1 to i-1 do SUM=SUM+L[i,k]*Y[k]; /* precision ( $\epsilon_s, \mu_s$ ) */
    Y[i]=B[i]-SUM; /* precision ( $\epsilon_s, \mu_s$ ) */
    /* Editor's note: After Cholesky factorization, where L[i,i] is not 1, must
    also divide by L[i,i] with precision ( $\epsilon, \mu$ ) */

/* Back Substitution: UX=Y */
for i=N downto 1 do
  begin
    SUM=0;
    for k=i+1 to N do SUM = SUM + U[i,k]*X[k]; /* precision ( $\epsilon_s, \mu_s$ ) */
    X[i]=(Y[i]-SUM)/U[i,i]; /* precision ( $\epsilon_s, \mu_s$ ) */
    /* Editor's note: in the line above, the division is done at precision ( $\epsilon, \mu$ ) */
  end
end

```

Application to libValidSDP

That concludes the excerpt of Demmel's 1981 technical report. Now we explain what's proved in libValidSDP.

Instead of Demmel's Proposition 2, we will be using the following lemma:

Rump/Jeannerod Lemma 2.1: Error analysis of $(c - \sum a_i b_i)/b_k$. (libvalidsdp.cholesky.lemma_2_1)
Let $\hat{y} = fl((c - \sum_{0 \leq i < k} a_i b_i)/b_k)$ computed by repeated subtraction left to right. Assume $b_k \neq 0$.
Then

$$|b_k \hat{y} - (c - \sum_{0 \leq i < k} a_i b_i)| < (k+1)\epsilon |b_k \hat{y}| \sum_{0 \leq i < k} |a_i b_i| + (1 + (k+1)\epsilon)(k+1 + |b_k|)\lambda\epsilon. \quad (\text{lemma2.1})$$

Error Analysis of Solving a Triangular System

Demmel Lemma 4, zero-based indexing. Consider the lower triangular system $Ly = b$ where L does not necessarily have ones on the diagonal. (The analyses when only ones are on the diagonal or L is upper triangular are similar.) Then in solving $Ly = b$ using the program in Appendix 0.2 (where inner products are accumulated with roundoff error ϵ_s and smallest number μ_s we obtain $\hat{y} = y + \delta y$ which actually satisfies $(L + \delta L)\hat{y} = b + \delta b$, where

$$|\delta L_{ij}| \leq |L_{ij}| \cdot \begin{cases} \epsilon & \text{if } i = j = 0 \\ i\epsilon_s & \text{if } i > j = 0 \\ (i+1-j)\epsilon_s & \text{if } i > j > 0 \\ \epsilon + \epsilon_s & \text{if } i = j > 0 \end{cases} \quad (0.80)$$

independent of underflow.

However, in all these cases we will prove a weaker bound because that allows us to re-use the Rump/Jeannerod Lemma 2.1 for $c - \sum x_i y_i$.

Theorem Demmel_L4_adapted:

$$|\delta L_{ij}| \leq |L_{ij}| n\epsilon \quad \text{and} \quad (\text{demmel_lemma_4})$$

$$\begin{aligned} \text{if } i = 0 \quad |\delta b_i| &\leq \lambda\epsilon |L_{00}| UN(y_0) & \text{if } y_0 \text{ underflows} \\ \text{if } i > 0 \quad |\delta b_i| &\leq (1 + (i+1)\epsilon)(i+1) \cdot \lambda_s \epsilon_s + & \text{intermediate underflows} \\ &(1 + (i+1)\epsilon)\lambda\epsilon |L_{ii}| UN(y_0) & \text{if } y_i \text{ underflows} \end{aligned} \quad (1)$$

The factor $(1 + (i+1)\epsilon)$ was omitted from Demmel's original but is required for correctness.

Proof. From libValidSDP lemma_2_1_aux, by a derivation of several lines. Q.E.D.

Solving a linear system by Cholesky+Substitution

Let $\hat{x}$ be the result of solving $Ax = b$ by Cholesky decomposition $A = R^T R$ followed by forward substitution $R^T y = b$ then back substitution $Rx = y$.

Lemma th_2_3_aux3: Accuracy of Cholesky decomposition. Suppose A is an $n \times n$ symmetric positive definite floating-point matrix, with $(n+1)\epsilon < 1$. Let $\hat{R}$ be the result of floating-point Cholesky decomposition (inner-product method) of A into $\hat{R}^T \hat{R}$. Let $\delta A = A - \hat{R}^T \hat{R}$. Let the vector R_j be the j th column of R . Let $a_{\max}$ be the maximum diagonal element of $|A|$. Then

$$|\delta A_{ij}| < (\min(i, j) + 2)\epsilon \|R_i\|_2 \|R_j\|_2 + 4\lambda\epsilon(n+1 + a_{\max}). \quad (2.3aux3)$$

Proof. See Rump+Jeannerod, or see libValidSDP. Q.E.D.

Remark. Let A and $\hat{R}$ be as above. Then

$$\|\hat{R}_j\|_2 \leq \sqrt{(A_{jj} + 2j\lambda\epsilon)/(1 - (j+2)\epsilon)}. \quad (2.3aux1)$$

Proof. LibValidSDP th_2_3_aux1.

Corollary. Combining (2.3aux1) and (2.3aux2), we have,

$$\begin{aligned}
|\delta A_{ij}| &< (\min(i, j) + 2)\epsilon \sqrt{(A_{ii} + 2i\lambda\epsilon)/(1 - (i + 2)\epsilon)} \sqrt{(A_{jj} + 2j\lambda\epsilon)/(1 - (j + 2)\epsilon)} + 4\lambda\epsilon(n + 1 + a_{\max}) \\
&\leq (n + 1)\epsilon \sqrt{(a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon)} \\
&\quad \cdot \sqrt{(a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon)} + 4\lambda\epsilon(n + 1 + a_{\max}) \\
&= (n + 1)\epsilon(a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon) + 4\lambda\epsilon(n + 1 + a_{\max})
\end{aligned}
\tag{ch_back_err}$$

Lemma. We obtain bounds on $\delta R^T \hat{R}$ (et cetera) by combining (2.3aux1) and (demmel_lemma_4):

$$\begin{aligned}
|(\delta R^T \hat{R})_{ij}| &\leq (|\delta R|^T |\hat{R}|)_{ij} \\
&\leq \sum_k (k + 1)\epsilon |\hat{R}_{ki}| \cdot |\hat{R}_{kj}| \quad (\text{by Lemma 0.4}) \\
&\leq n\epsilon \|\hat{R}_i\|_2 \|\hat{R}_j\|_2 \quad (\text{by Cauchy - Schwartz}) \\
&\leq n\epsilon \sqrt{(A_{ii} + 2i\lambda\epsilon)/(1 - (i + 2)\epsilon)} \sqrt{(A_{jj} + 2j\lambda\epsilon)/(1 - (j + 2)\epsilon)} \quad (2.3aux1) \\
&\leq n\epsilon \sqrt{(a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon)} \sqrt{(a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon)} \\
&= n\epsilon(a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon)
\end{aligned}
\tag{csolve_back_err_aux1}$$

Lemma. Bounds on $\delta R^T \delta R$ as follows:

$$\begin{aligned}
|(\delta R^T \delta R)_{ij}| &\leq (|\delta R^T| \cdot |\delta R|)_{ij} \\
&\leq \sum_k |\delta R^T|_{ik} |\delta R|_{kj} \\
&= \sum_k |\delta R|_{ki} |\delta R|_{kj} \\
&\leq \sum_k (k + 1)^2 \epsilon^2 |\hat{R}_{ki}| \cdot |\hat{R}_{kj}| \quad (\text{csolve_back_err_aux2}) \\
&\leq n^2 \epsilon^2 \|\hat{R}_i\|_2 \|\hat{R}_j\|_2 \\
&\leq n^2 \epsilon^2 (a_{\max} + 2(n - 1)\lambda\epsilon)/(1 - (n + 1)\epsilon).
\end{aligned}$$

Theorem: Cholesky factor + substitution. Suppose A is an $n \times n$ symmetric positive definite floating-point matrix with minimum eigenvalue $> \sqrt{\lambda}$, with $(n + 1)\epsilon < 1$. Suppose b is a floating-point vector of length n . Suppose $\hat{x}$ is the result of floating-point Cholesky factorization of A into $\hat{R}^T \hat{R}$ followed by forward substitution finding $\hat{y}$ such that $\hat{R}^T \hat{y} = b$ and then $\hat{R}\hat{x} = \hat{y}$. Then

fill this in ... (xxx)

Proof. If it were not for the underflow terms δb and δy , we would have

$$\begin{aligned}
b &= (\hat{R}^T + \delta R^T) \hat{y} \\
&= (\hat{R}^T + \delta R^T)(\hat{R} + \delta R) \hat{x} \\
&= (\hat{R}^T \hat{R} + \delta R^T \hat{R} + \hat{R}^T \delta R + \delta R^T \delta R) \hat{x} \quad (\text{csolve_back_err}) \\
&= (A + \delta A + \delta R^T \hat{R} + \hat{R}^T \delta R + \delta R^T \delta R) \hat{x} \\
&= A\hat{x} + (\delta A + \delta R^T \hat{R} + \hat{R}^T \delta R + \delta R^T \delta R) \hat{x}.
\end{aligned}$$

From (2.3aux3) we have $|\delta A_{ij}| < (\min(i, j) + 2)\epsilon \|\hat{R}_i\|_2 \|\hat{R}_j\|_2 + 4\lambda\epsilon(n + 1 + a_{\max})$.

From

Appendix 0.2: Program Listing of Solution of Both Triangular Systems Following Triangular Decomposition

```

array B[N] of real;
array X[N] of real;
array Y[N] of real;
/* Again, B,X, and Y could occupy the same locations in memory. Arrays
A,L and U are defined above. As in Appendix 1, precisions of all computations
except entries of B, X, and Y are shown. */

```

```

/* Forward Substitution: LY = B */

```

```

for (i=0; i<N; i++)
  begin
    SUM=0;
    for (k=i; k<i-1; k++) do SUM=SUM+L[i,k]*Y[k]; /* precision  $(\epsilon_s, \mu_s)$  */
    Y[i]=B[i]-SUM; /* precision  $(\epsilon_s, \mu_s)$  */
    /* Editor's note: After Cholesky factorization, where L[i,i] is not 1, must
       also divide by L[i,i] with precision  $(\epsilon, \mu)$  */

/* Back Substitution: UX=Y */
for (i=N-1; i>=0; i-) do
  begin
    SUM=0;
    for (k=i+1; k<N; k++) do SUM = SUM + U[i,k]*X[k]; /* precision  $(\epsilon_s, \mu_s)$  */
    X[i]=(Y[i]-SUM)/U[i,i]; /* precision  $(\epsilon_s, \mu_s)$  */
  /* Editor's note: in the line above, the division is done at precision  $(\epsilon, \mu)$  */

```